Exp no:1
23/7/26

Basic Networking Commands

1) arp -a

Displays the ARP, cache, showing the IP address and corresponding MAC Address of devices on the local network.

O/P:

-gateway (10.0.2.2) at 52:54:00:12:35:02 [ether] on enp0s3

2) hostname

display the name of our computer.

O/P:

localhost.localdomain

3) ipconfig /all

Displays complete TCP/IP network configuration, including IP address, subnet mask, gateway, DNS, DHCP and MAC address.

O/P:

windows IP configuration

Host name.....: AVINASH

Primary Dns Suffix.....

Node Type . . . . . : Mixed

IP Routing Enabled . . . . : No

WINS Proxy Enabled . . . . : No

4) nbstat -a

Displays the NetBIOS name table of a remote computer to help troubleshoot NetBIOS name resolution.

O/P:

Displays protocol statistics and current TCP/IP connections using NBT.

NBTSTAT [[-a RemoteName] [-A IP address] [-c] [-n] [-r] [-R] [-RR] [-s] [-S] [interval]

-a (adapter status) Lists the remote machine's name table given its name

-A (Adapter status) List the remote machine's name table given its IP address.

-n (names) Lists Local NetBIOS names.

5) netstat

Displays active network connections, listening, ports and network statistics.

O/P:

Active Connection

Proto	Local Address	Foreign Address	State
TCP	[::1]:1521	AVINASH:49672	ESTABLISHED
TCP	[::1]:49672	AVINASH:1521	ESTABLISHED

6) netstat -r

Displays the routing table of the computer.

Interface List:

21...00 ff eb da 62 66....TAP-Windows Adapter V9

6...3e 8d 3d 7f 5d 30...Realtek RTL8851BE WiFi 6 802.11ax PCIe Adapter #3

3...3a 8d 3d 7f 5d 30...Realtek RTL8851BE WiFi 6 802.11ax PCIe Adapter #5

2...38 8d 3d 7f 5d 30...Realtek RTL8851BE WiFi 6 802.11ax PCIe Adapter

1.....Software Loopback Interface 1

7) nslookup

Finds the IP address of a domain name using the DNS servers.

O/P:

Default server: Unknown

Address: 127.0.0.1

8) pathping

Traces the route to a destination & measures packet loss at each network hop.

O/P:

usage: pathping [-g host-list] [-h maximum_hops]
[-i address] [-n] [-p period]
[-q num_queries] [-w timeout]
[-4] [-6] target_name

9) ping

Test connectivity b/w, your computer & another computer or website using ICMP.

O/P:

ping [-t] [-a] [-n count] [-l size] [-f]
[-i TTL] [-v TOS] [-r count] [-s count]

[-j host-list] [-s srcaddr] [-c compoodtmerdt]

[-p] [-4] [-6] target-name

10) route

Displays the IP routing table used by the OS to route network traffic.

O/P:

ROUTE [-f] [-p] [-4|-6] Command [destination]

[MASK netmask] [gateway] [METERIC metric]

[IF interface]

Linux Networking Commands

1) ip

Display IP address

Configure network interface

Add / Delete IP address

Manage routing tables

ii) Display IP Address:

ip

Displays all network interfaces and their assigned IP addresses.

O/P:

usage: ip [OPTIONS] object {command | help}

ip [-force] -batch filename

1. lo: <LOOPBACK, UP, LOWER_UP> mtu 65536 qdisc noqueue state unknown group default qlen 1000

inet 127.0.0.1

2. enp0s3:

inet 10.0.2.15/24

iii) ip link show

Displays the status of all network interfaces.

O/P:

1. lo: <LOOPBACK, UP, LOWER_UP>

2. enp0s3: <BROADCAST, MULTICAST, LOWER_UP>

iii) ip route:

Displays the routing table & default gateway

O/P:

10.0.2.2 dev enpV03 proto dhcp src 10.0.2.15 metric 100 10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100

2) ip address add

Assigns a new IP address to a network interfaces.

Ex:

ip address add 192.168.1.254/24

O/P:

1. lo: <LOOPBACK, UP, LOWER_UP>

2. enp0s3: <BROADCAST, MULTICAST, UP, LOWER_UP>

inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3

inet 192.168.1.254/24 scope global enp0s3

3) Delete an IP address:

Remove the IP address from the interface

Eg: ip address del 192.168.1.254/24 dev enp0s3

O/P:

1. lo: <LOOPBACK, UP, LOWER_UP>

2. enp0s3: <BROADCAST, MULTICAST, UP, LOWER_UP>

inet 10.0.2.15/24 brd 10.0.2.255

scope global dynamic enp0s3

4. Bring Interface Online:

Enables the Network Interface:

Eg: ip link set enp0s3 UP

O/P:

1.10: mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT mode group default qlen 1000 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00

ipconfig

It is a staple in many sysadmin's tool belt for configuration configuring & Troubleshooting Networks.

O/P:

Windows IP configuration

Unknown adapter MCAfeeVPN:

Media State . . . . . : Media disconnected

Connection-specific DNS Suffix:

Wireless LAN adapter WiFi 3:

Media State . . . . . : Media Disconnected

Connection specific DNS suffix:

mtr:

Combines the functionality of the ping and Trace route commands.

as mtr google.com

Shows you the statistics, including each hop with time & loss%.

O/P:

Host	Loss%	snt	Last	Avg	Best
172.16.8.1	0.0%	134	0.1	0.2	0.1
172.97.63.129	2.6%	134	3.9	5.2	1.1
???	2.2%	188	46.6	18.6	4.1

b) mtr -g google.com

Show numeric IP address

c) mtr -b google.com

Show numeric IP address & hostnames.

d) mtr -c 10 google.com

Set the number of ping that you want.

13) tcdump

This command is designed for capturing & displaying packets.

i > tcpdump -i eth0 -c 10 host 8.8.8.8

Filter our traffic comming from a specific host.

13) ping AVINASH

It display best way to test connectivity b/w two nodes.

Syntax: ping hostname

O/P:

pinging AVINASH [fe80::521f:7690:f461:C631-2

with 32 bytes of data.

Reply from fe80::521f:7ba0:f461:c631-2:

time<1ms

ping statistics for fe80::521f::7ba0:f461:C63

packets: sent=4, Recieved=4, Lost=0 (0% Loss)

Approximate round trip times in milli-seconds:

Minimum=0ms, Maximum=0ms, Average=0ms

Result:

The Basic Networking & Linux Commands are learned & executed successfully.

Exp.no:2
27/7/26

Study of Different Types of Network Cables

A) Understand different types of Network Cable

Different types of cables used in networking are

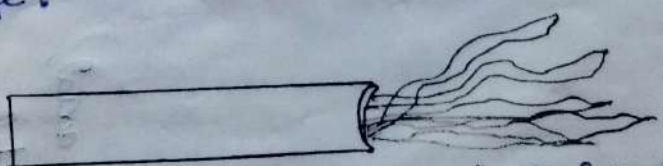
1) Unshielded Twisted Pair (UTP)
2) Shielded Twisted Pair (STP)
3) Coaxial Cable
4) Fibre Optic Cable

1) Cable Type:

UTP

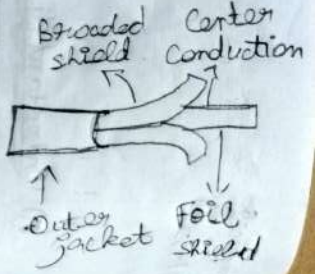
Category	Maximum Data Transmission	Advantages/ Disadvantages	Application /use
Category 3	10 bps	Advantage: Cheaper in cost	10 Base - T Ethernet
Category 5	up to 100 Mbps	Easy to install as they a smaller overall diameter	Fast Ethernet, Gigabit Ethernet
Category 5e	1 Gbps		Fast Ethernet, Giga Ethernet

Image:



↓ Outer Jacket

↳ Twisted pair with color coded plastic simulation

Cable Type	Category	Maximum Data Transmission	Advantages/ Disadvantages	Application/ Use	Image
STP	Category 6, 6a	10 Gbps	Advantages: Shielded Faster than UTP Less Susceptible to noise & Interference. Disadvantages: Expensive Greater Installation Effort	Gigabit Ethernet 10G Ethernet (55m) widely used in Data Center. Gigabit Ethernet, 10G Ethernet (100m)	
	Category 7	10 Gbps			
Co-axial Cable	RG-9 RG59 RG11	10 - 100 Mbps	High Bandwidth Immune to Interference Low loss Bandwidth	Speed of signal is 500m Television network high	

Cable Type	Category	Maximum Data Transmission	Advantage / Disadvantage	Application / use	Image
Fibre optics Cable	Single Mode Multi-Mode	100 Gbps	Disadvantages: Limited distance cost size is bulky Adv: High speed high Bandwidth High security Disadvantages: Expensive Requires skilled Installers	Maximum Distance of fibre optics Cable in around 100 metres	

4 b) Make Your Own Ethernet Cross-Over Cable / Straight Cable.

Tools and parts needed:

Ethernet Cabling: CAT5e is certified for gigabit support but CAT5 cabling works as well, just over shorter distances.

A crimping Tool: This is an all in one networking tool shaped to push Down the pins in the plug and strip and cut the shielding off the cables.

Two RJ45 plugs

Optional two plug shields

Straight thru Cable

X- Over Cable

Note: Next Page Diagram

Step:

1) To start construction of the device, begin by threading shields onto the cable.

2) Next, strip approximately 1.5 cm of cable shielding from both ends. The Scrimping tool has a round area to complete this Task.

3) After you will need to untangle the wires, there should be four "Twisted pairs". Referencing back to the sheet, arrange them from top to bottom. One end should be in arrangement A & the other in B.

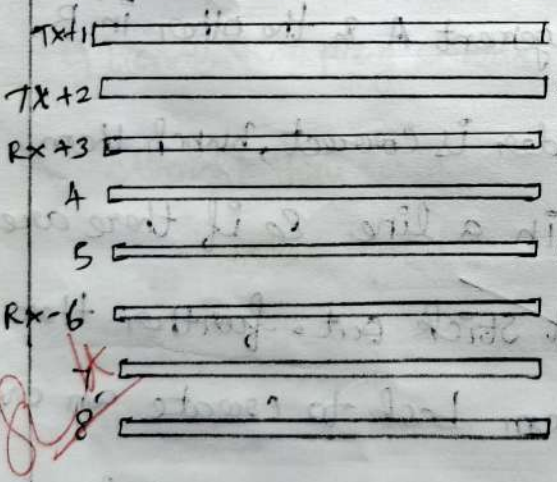
4) One the order is correct, bunch them together in a line & if there are any that stick out further than other snip them back to create an even level. The difficult aspect is placing these into the RJ45 plug without meeting up the order.

5) Next, push the Cable right in the notch at the end of the plug needs to be just over the Cable shielding and if isn't that means that you stripped off too much shielding.

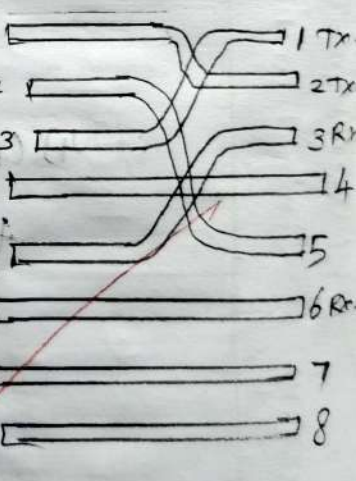
6) After the wires are securely sitting inside a plug, insert it into the crimping tool and push down.

Lastly, repeat for the other end using Diagram B.

Straight thru Cable



X-over Cable



Result:

Different types of cable network are learned & Execute successfully.

Exp: 3
27/7/26

CISCO PACKET TRACER

a) Analyze the behaviour of network device using CISCO PACKET TRACER Simulator

1. From the network component box, click & drag and drop the below components.

   a. ~~A~~ Generic PC's & One HUB

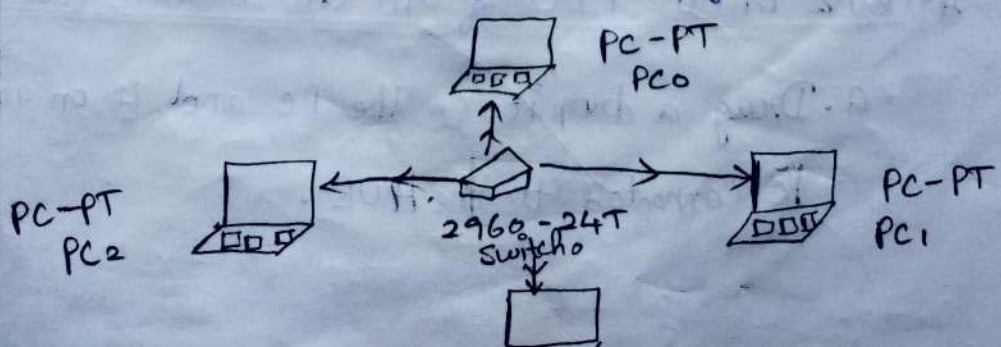
   b. 4 Generic PC's & One Switch

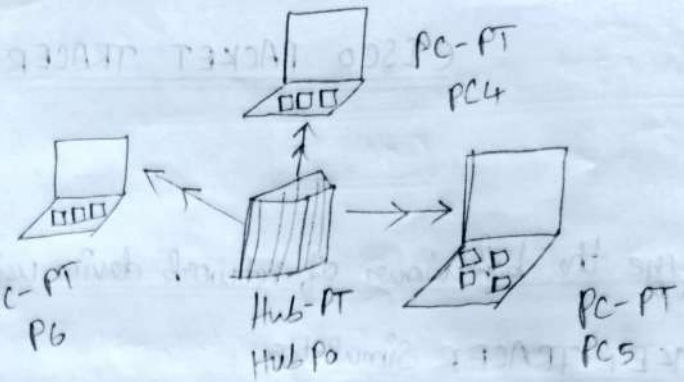
2) Click on Connections

* click on copper straight - through cable.
* Select One of the PC and connect it to HUB using the cable. The link LED should glow in green, indicating that the link it up.

Similarly connect remaining 3 PC's to the HUB.

* Similarly connect 4 PC's to the switch using copper, Straight - through cable.





3) Click on the PC's connected to hub, go to the Desktop tab, click on IP configuration, and enter an IP address and subnet mask. Here, the deafault gateway and DNS server information is not needed as there are only two end device in the network.

PC0	PC1
IP Configuration	IP Configuration
IP configuration	IP configuration
○ DHCP ◉ Static	○ DHCP ◉ Static
IP Address 10.1.1.1	IP Address 10.1.1.2
Subnet mask 255.0.0.0	Subnet MASK 255.0.0.0
Default gateway	Default gateway
DNS Server	DNS Server

4. Click on the PDU from the common tool bar

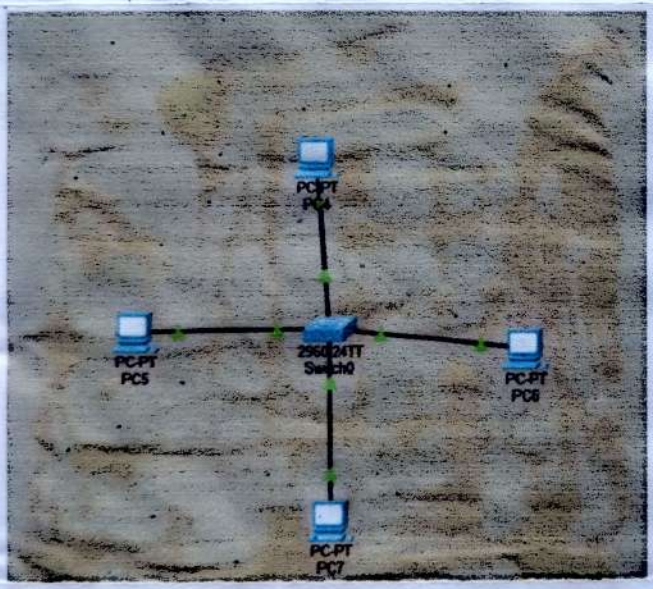
a. Drag a drop it on the PC and it on another PC connected to the HUB.

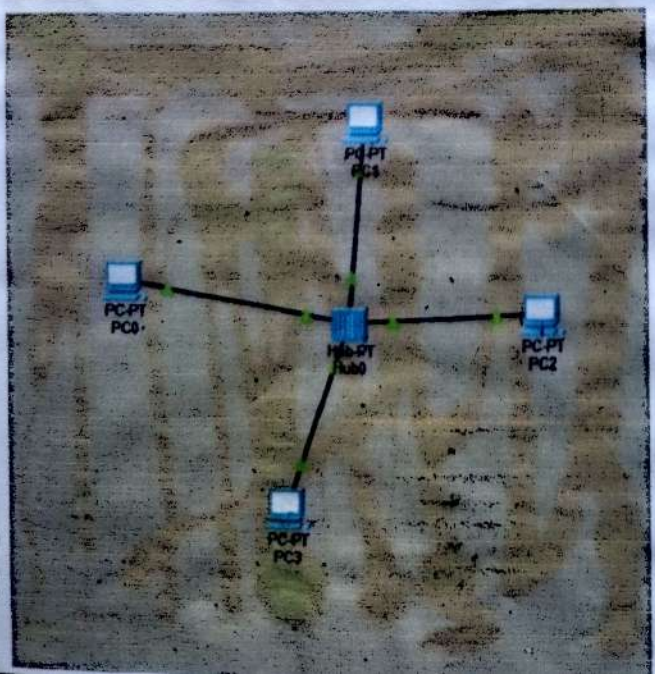
5. Observe the flow of PDU from source PC to destination PC by selecting the Realtime mode of simulation.

6) Repeat Step #3 to Step #5 of the PC's connected to the switch.

RESULT:

CISCO PACKET TRACER are learned & executed successfully.

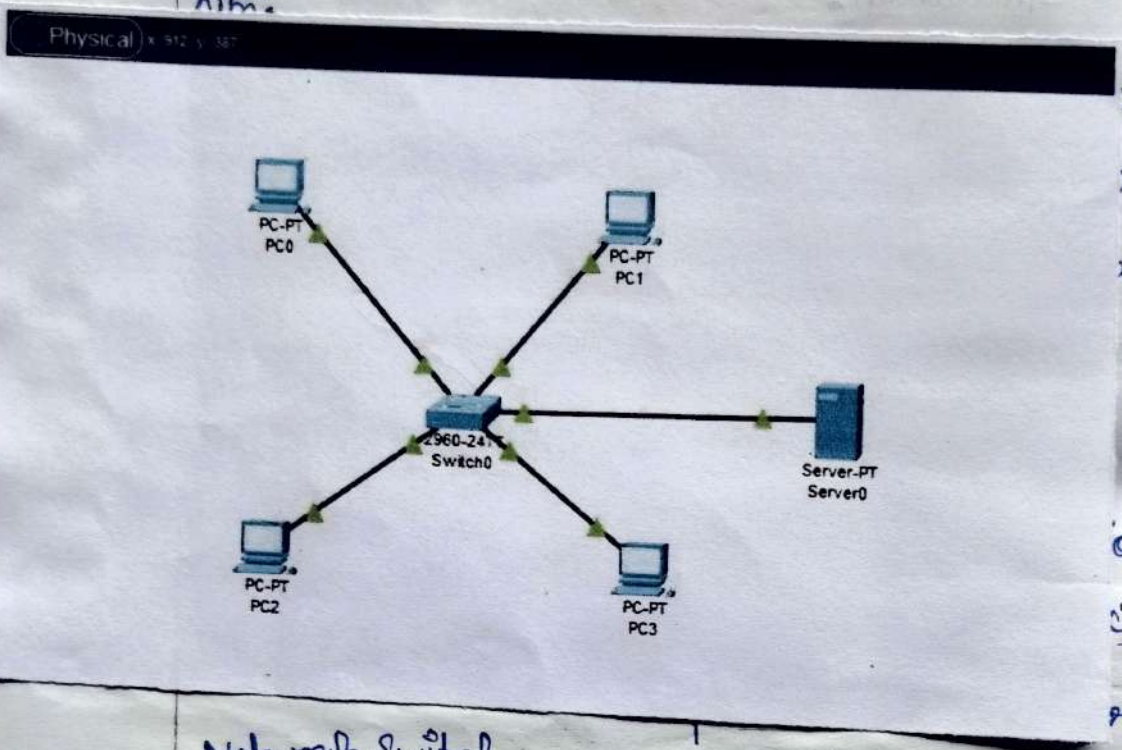




Exp No: 4
03/08/26

SETTING UP A LOCAL AREA NETWORK (LAN)

Aim:

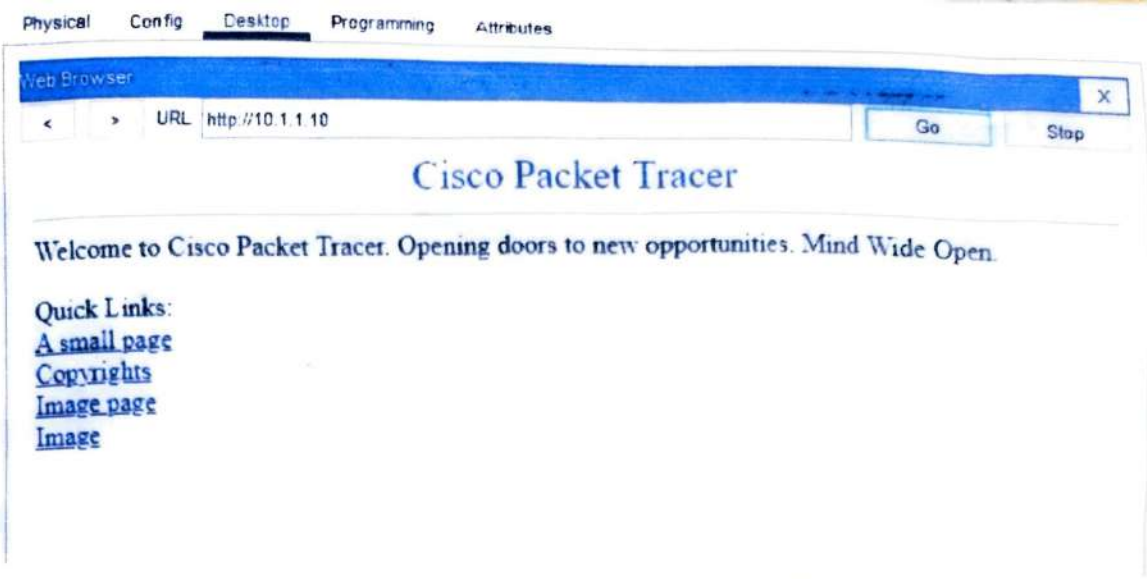


Network Switch

Ethernet Cables (Cat5e/Cat6)	4	Straight through RJ-45 connectors

IP address configuration table

Device	IP Address	Subnet mask	Connected to
PC0	10.1.1.1	255.0.0.0	Switch port Fa0/1
PC1	10.1.1.2	255.0.0.0	Switch port Fa0/2
PC2	10.1.1.3	255.0.0.0	Switch port Fa0/3
PC3	10.1.1.4	255.0.0.0	Switch port Fa0/4
Switch	10.1.1.5	255.0.0.0	VLAN 1 Interface



2) Cable the devices

→ Click Connections → select Copper Straight-through Cable.

→ Click each PC, then click the switch and select an available Fast Ethernet port to complete each connection.

→ Repeat for all four PC's (ports Fa0/1 to Fa0/4)

Green link lights confirm a successful physical connection.

3) Assign IP Addresses to PC's:

→ Click a PC → Desktop Tab → IP Configuration

→ Enter the Static IP Address and subnet Mask as per the IP table.

4) Configure the switch Management IP:

→ CLI tab

→ Click the switch ~~Management IP~~

-enable
configure terminal
interface Vlan
ip address 10.1.1.5 255.0.0.0
no shutdown
exit

5) Verify connectivity using ping

→ Click a PC → Desktop tab → and prompt
→ Use the ping to test connectivity
→ successful message

6) Demonstrate Data Sharing (Server simulation)

→ Add a server device (End devices → Server - PT) and connect it to the switch.
→ Assign it an IP Address
→ Open the server's services tab & enables the HTTP.
→ From another PC, open web Browser and enter the server's IP address to load the page
Confirmation: successful data transfer across the LAN.

Result:

This LAN is settled up and connection is verified successfully.

EXPERIMENT ON PACKET CAPTURE TOOL WIRESHARK

Aim:

Experiments on Packet Capture Tool Wireshark

Packet Sniffer:

- Sniffs messages being sent / recieved from/ by your computer.
- Store & display the contents of the various protocol fields in the messages.
- Passive Program
  - i) Never sends packets itself
  - ii) No packets address to it.
  - iii) Recievers a copy of all packets (sent/recieved)

Packet Sniffer Structure Diagnostic tool:

- Tcpdump - Eg: tcpdump -enx host 10.12.9.41.
  one 3.out
- wireshark - wireshark -nr exe3.out

Procedure:

- Select the local area connection in wireshark
- Go to capture option.

* Select capture and stop automatically after 100 packets.

* Then click start Capture.

* Save packets.

i) Create a filter to display only TCP [redacted] UDP packets. Inspect the packets and provide the flow graph.

* Go to search bar at the top of the tool.

* Search TCP packets in search bar.

* To see flow graph click Statistics flow graph.

* Save the packets.

ii) Create a filter for ARP packets

* Go to search option

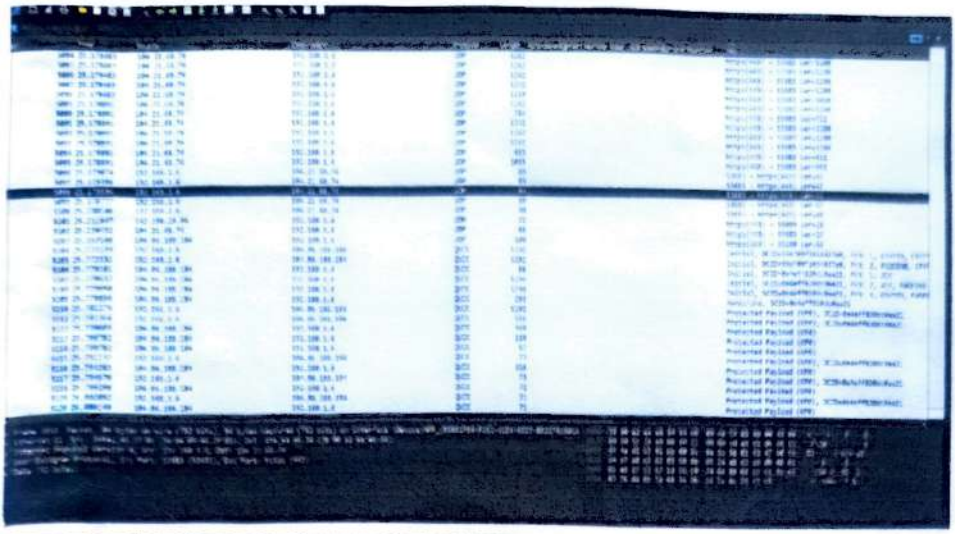
* Search ARP packet in search bar

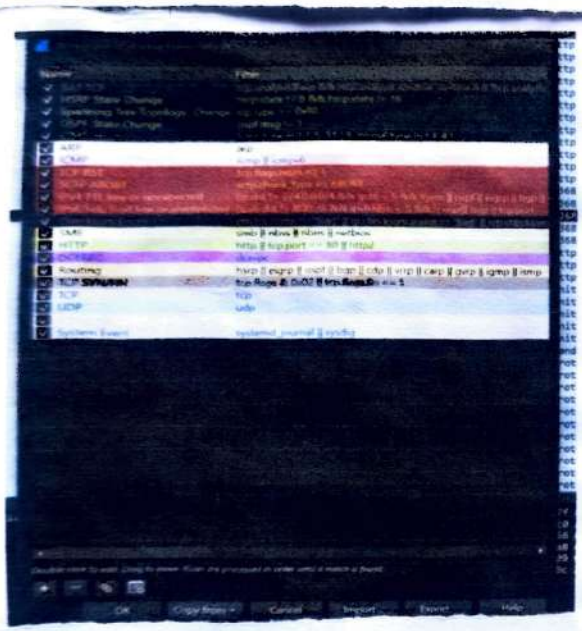
* Save the packets.

iii) Creating a filter to display DNS packets and provide the flow graph.

iv) Creating a filter to display HTTP only packet and inspect the packet.

v) Create filter for ICMP and DHCP packets and inspect and show the packets









Result:

Wireshark helps us in analysing and understanding our network traffic and packet flow through various filters and flow graphs.

Ex-6
13/8/26

PYTHON IMPLEMENTATION OF HAMMING CODE CONCEPT

Error Correction at Data Link Layer (AIM):

→ Hamming code is a set of error correction codes that can be used to detect and correct the errors. when the data is transmitted from the sender to the Reciever. To implement error detection using Hamming code technique in Data Link layer.

SENDER PROGRAM:

→ Takes text input.

→ Converts text to binary

→ Divides binary data into 4 bit groups

→ Adds 3 redundant parity bits using Hamming Code.

→ Saves the encoded data in channel.text.

Code:

```
def hamming_encode(data):
    encoded = ""
    for i in range(0, len(data), 4):
        d = data[i:i+4]
    if len(d) < 4:
        d = d.ljust(4, '0')
    d1, d2, d3, d4 = map(int, d)
    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p4 = d2 ^ d3 ^ d4
    code = str(p1) + str(p2) + str(d1) + str(p4) +
           str(d2) + str(d3) + str(d4)
    encode += code
    return encoded
text = input("Enter text: ")
binary = ' '.join(format(ord(c), '08b') for c in
                  text)
print("Binary data:", binary)
encoded_data = hamming_encode(binary)
with open("channel.txt", "w") as file:
    file.write(encoded_data)
```

print ("Hamming encoded data: ", encoded_data)

print ("Data saved to channel.txt")

RECIEVER PROGRAM:

→ Reads data from channel.txt

→ checks every 7-bit hamming code

→ Find the error position using syndrome

→ Corrects the errors & removes parity bits

→ Converts binary back to ASCII text

CODE:

```
def hamming_decode(data):
    corrected = ""
    error_found = False
    for i in range(0, len(data), 7):
        block = list(map(int, data[i, i+7]))
        if len(block) < 7:
            continue
        p1, p2, d1, p4, d2, d3, d4 = block
        s1 = p1 ^ d1 ^ d2 ^ d4
        s2 = p2 ^ d1 ^ d3 ^ d4
        s4 = p4 ^ d2 ^ d3 ^ d4
        error_position = s1 + (s2*2) + (s4*4)
```

```
if error_position != 0:
    error_found = True
    actual_position = i + error_position
    print("Error detected at bit position:",
          actual_position)
    block[error_position - 1] ^= 1
    print("Error corrected at bit position",
          actual_position)
    Corrected += str(block[2])
    corrected += str(block[4])
    Corrected += str(block[5])
    Corrected += str(block[6])
return corrected, error_found
with open("channel.txt", "r") as file:
    Channel_data = file.read().strip()
print("Data Recieved from channel:")
print(channel_data)
binary_data, error_found = hamming_decode
                           (channel_data)
text = ""
for i in range(0, len(binary_data), 8):
    byte = binary_data[i, i+8]
```

```
    if len(byte)==8:
        text += chr(int(byte,2))
print("\n Corrected binary data:")
print(binary_data)
if error_found:
    print("\n Error was detected & corrected").
else:
    print("\n No error detected.")
print("Recieved text:",text)
```

INPUT:

Enter the text : HELLO

OUTPUT:

Binary data: 01001000 01000101 01001100 01001100 01001111

Hamming encoded data: 10011100 11100000 10011000 10010111 00111000 11110011 00111111

Data saved to channel.txt

RESULT:

Thus, the hamming code was successfully implemented to detect and correct errors in the transmitted data.

Ex-7
24/8/26

SLIDING WINDOW PROTOCOL

AIM:

write a program to implement flow control at data link layer using sliding window Protocol. Simulate the flow of frames from one node to another.

SENDER PROGRAM:

```
import time
w = int(input("window size: "))
msg = input("message: ")
frames [(i,c) for i,c in enumerate(msg)]
i=0
while i< len(frames):
    end = min(i+w, len(frames))
    with open("Sender_Buffer.txt","w") as f:
        for n,c in frames[i:end]:
            print("Frame",n,":",c)
            f.write(f"{n},{c}\n")
    time.sleep(2)
with open("Reciever_Buffer.txt") as f:
    ack = f.read().strip()
```

```
if ack.starts with ("ACK:"):
    a = int (ack [4:])
    if a==end:
        i==end
    else
        print("wrong ACK-Resending")
elif ack.startswith ("NACK"):
    i= int (ack [5:])
    print ("NACK-Resending from ",i)
print ("Transmission Complete")
```

RECIEVER PROGRAM:

```
import time
import os
expected = 0
while True:
    if not os.path.exists("Sender_Buffer.txt"):
        time.sleep(1)
        continue
    with open ("send_Buffer.txt") as f:
        frames = f.readlines()
    if not frames
        time.sleep(1)
```

```
error = False

for line in frames:
    n, data = line.strip().split(",")
    n = int(n)
    print("Recieved:", n, data)

if n != expected
    with open("Reciever_Buffer.txt", "w") as f:
        f.write("NACK:" + str(expected))
    error = True
    break
expected += 1

if not error:
    with open("Reciever_Buffer.txt", "w") as f:
        f.write("ACK" + str(expected))

open("sender_Buffer.txt", "w").close()

time.sleep(1)
```

SENDER CODE INPUT:

window size = 3

Message: HELLO

SENDER CODE OUTPUT:

Frame 0 : H

Frame 1 : E

Frame 2 : L

Frame 3 : L

Frame 4 : O

RECIEVER CODE INPUT:

Sender_Buffer.txt

0, H
1, E
2, L
3, L
4, O

RECIEVER CODE OUTPUT:

Recieved : 0 H

Recieved : 1 E

Recieved : 2 L

Recieved : 3 L

Recieved : 4 O

RESULT:

The sliding window Protocol was successfully implemented to control frame flow and handle

Exp.no:8
28/9/26

NMAP to Discover Live Host Using Nmap Scans

AIM:

To discover live hosts on a network using Nmap host discovery scans - ARP scan, ICMP scan, and TCP/UDP ping scan - and to understand which protocol operates at which layer of the TCP/IP Model and when each one applies.

1. ARP Host Discovery

The ARP scan successfully identified the active hosts in the local subnet. ARP request were broadcast to the network and the live hosts responded with their MAC address.

This confirms that ARP based discovery works at the Link Layer and is mainly used for discovering hosts on the same local network.

2) ICMP Echo Scan (-PE)

The ICMP Echo scan identified hosts that responded to ICMP Echo Requests. The hosts that replied were considered Active.

3) ICMP Timestamp Scan (-PP)

The ICMP TimeStamp scan identified some time hosts that responded to timestamp Requests some hosts didn't responded because ICMP timestamp packets may be filter by firewalls.

4) ICMP Address Mask Scan (-PM)

The ICMP Address Mask Scan recieved few or no responses. This is because Address Mask Requests are commonly blocked or unsupported by modern Systems and firewalls.

5) TCP SYN. Ping(-PS)

The TCP SYN ping identified active hosts by sending TCP SYN packets to the specified port.

An SYN/ACK response from an open port or an RST response from a closed port or an RST response.

6) TCP ACK ping(-PA)

The TCP ACK ping identified active hosts by sending TCP packets with the ACK flag set. Send the packets doesnot belong to an existing connection, a live host generally responds with an RST packet.

7) UDP ping:

The UDP ping identified active hosts by sending UDP packets to the specified port.

when the target port is closed, the host normally responds when an ICMP port unreachable message confirming that hosts is active.

RESULT:

The NMAP host discovery experiments were successfully performed using ARP, ICMP, TCP and UDP scanning Techniques.

Exp.no:9
Date: 17.9.26

IMPLEMENTATION OF SUBNETTING IN CISCO PACKET TRACER SIMULATOR

AIM:

To implement classless IP subnetting and test network connectivity using CISCO packet Tracer.

PROCEDURE:

Open CISCO packet Tracer and create a new network topology.

Add the required Routers, Switches and PC's to the workspaces.

Connect the devices using appropriate cables.

Dividing the network 192.168.1.0/24 into smaller subnets using a /27 subnet mask.

Assign suitable IP addresses and subnet masks to the router interfaces.

Configure the PC's with their respective IP address subnet mask & default gateway.

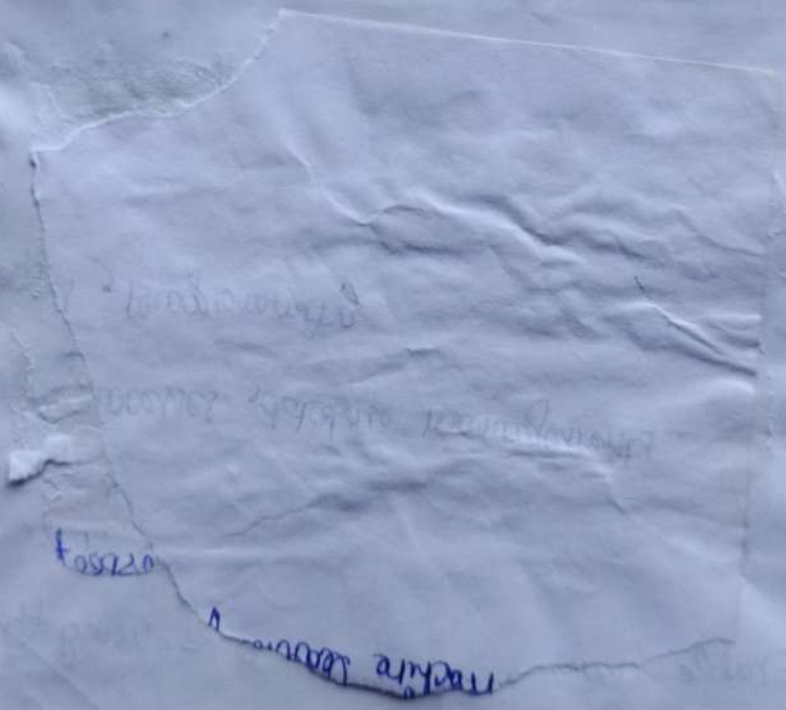
Enable the Router Interfaces using the no shutdown command.

Using ping command from the PC's to test connectivity.

Verify that the packets are successfully transmitted between the interfaces.

CLI Command:

#enable
Configure Terminal
interface FastEthernet 0/0
ip address 198.168.1.10 256.255.255.0
no shutdown
#exit



Result:

Thus, classless IP subnetting was successfully implemented and tested using CISCO packet Tracer.

Exp no: 10d)
Date: 24.9.26

INTERNETWORKING USING ROUTER

AIM:

To implement and design configure a simple internetwork using a router and two PC's in CISCO packet tracer and verify network connectivity.

Procedure:

Open CISCO Packet Tracer and create a new network topology.

Place one Router and 2 PC's in the workspace.

Connect PC0 and PC1 to the router using copper straight through cables.

Configure Router 1 Interfaces with the following

FastEthernet 0/0 → 192.168.10.1/24

FastEthernet 0/1 → 192.168.20.1/24

Enable both router interfaces using the no shutdown command.

Configure PC0 with

IP Address → 192.168.10.2

Subnet Mask → 255.255.255.0

Default gateway → 192.168.10.1

Configure PC1 with

Which TCP ping scan does not require a privileged account?

-sT

✓ Correct Answer

Which TCP ping scan requires a privileged account?

-sS

✓ Correct Answer

What option do you need to add to Nmap to run a TCP SYN ping scan on the telnet port?

-PS23

✓ Correct Answer

What is the option required to tell Nmap to use ICMP Timestamp to discover live hosts?

-PP

✓ Correct Answer

What is the option required to tell Nmap to use ICMP Address Mask to discover live hosts?

-PM

✓ Correct Answer

What is the option required to tell Nmap to use ICMP Echo to discover live hosts?

-PE

✓ Correct Answer

PC1

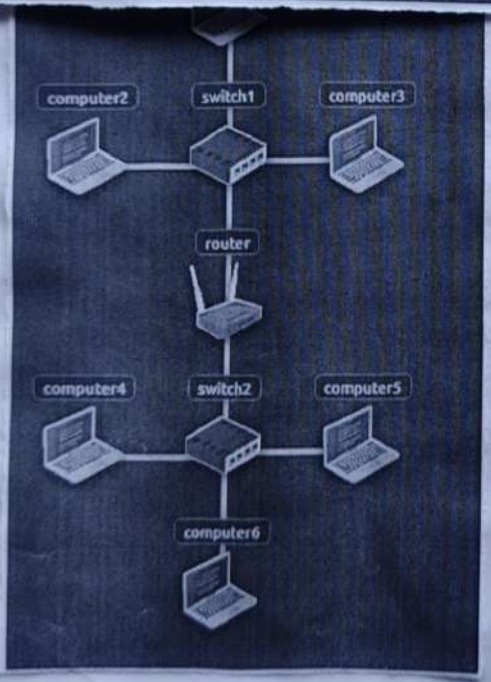
successfully and an acknowledgement is Recieved.

Answer the questions below

How many hosts are found alive after scanning the CONNECTION_IP/24?

3

✓ Correct Answer



Result:

A simple Internetwork using a Router and two PC's was successfully designed & configure.

Exp no: 10(b)
Date: 24.9.26

Wireless Router + DHCP + Internet

AIM:

To design & configure an internetwork using a wireless Router, DHCP Server & Internet cloud and Verify Network Connectivity.

Procedure:

Open Cisco Packet Tracer and create a new network topology.

Add a wireless Router, PC, Laptop, Cable modem Internet cloud and CISCO.com Server.

Connect the device using Appropriate Copper Straight through & Coaxial Cables.

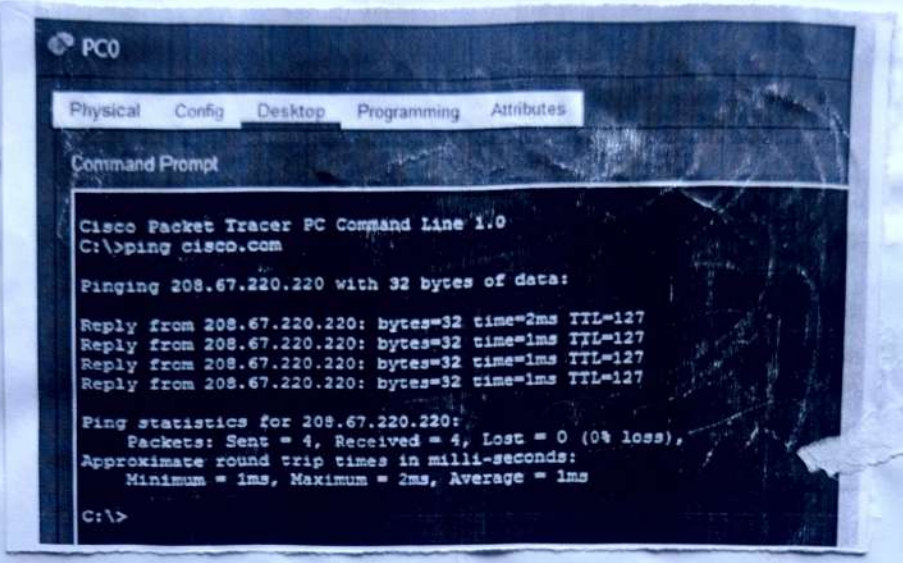
Configure the device using Appropriate cable. IP Address 192.168.0.1 and enable the DHCP Server.

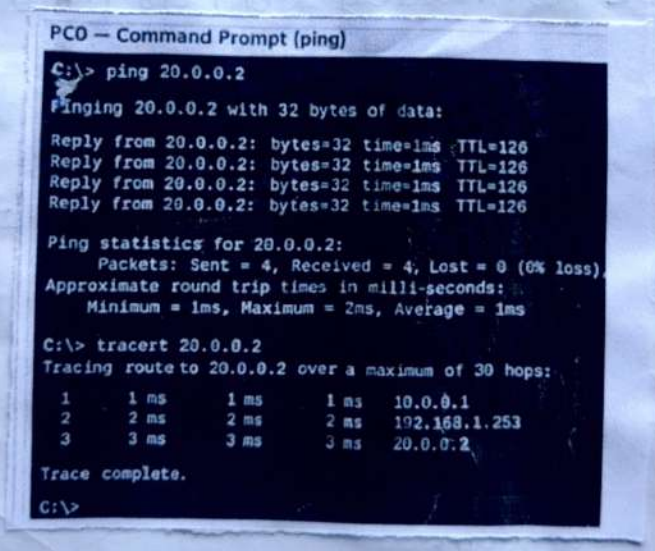
Set the wireless Network Name (SSID) as Home Network.

Configure the laptop with a wireless Module & the PC to obtain its IP Address automatically using DHCP.

Enable DHCP and DNS services on the cisco.com server.

Test connectivity by using the command ping cisco.com.





Result:

A Internetworking using wireless Router, DHCP server & Internet Cloud was successfully designed.

Exp: 11)a)
Date: 5.10.26

Exp-11
ROUTING

11)a) Static Routing:

AIM:

To Simulate and configure static routing using cisco packet Tracer.

OBSERVATION:

1) Static Routing Involves Manually adding routes to a router's Routing Table.

2) Routers Automatically know the networks directly connected to their interfaces.

3) Routes are manually added for Networks that are available through other Routers.

4) Both main & backup Routes can be configured for a destination.

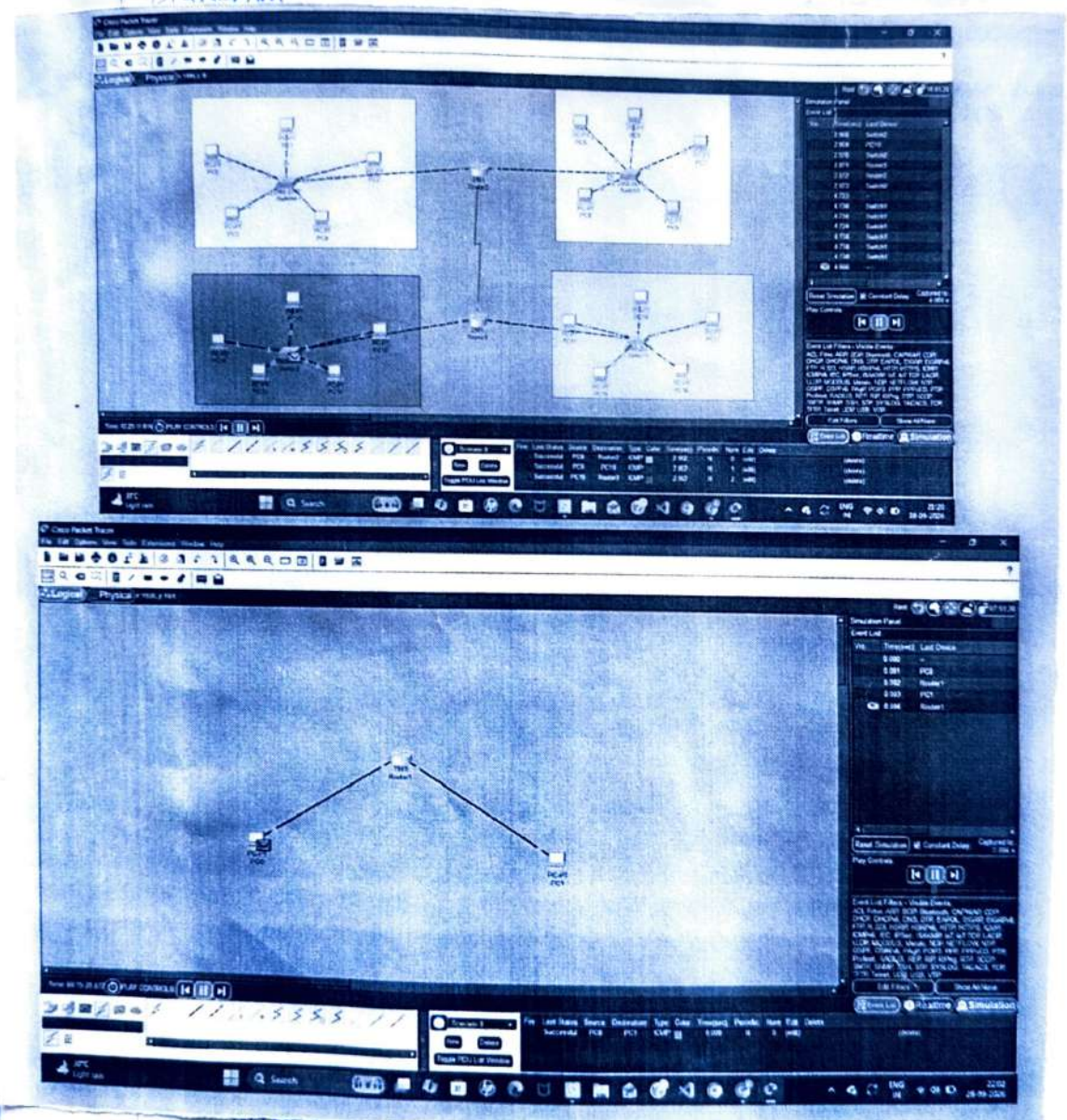
5) Administrative Distance (AD) is used to select the preferred static Route.

6) The Routing Table is checked using show ip route static.

7) Connectivity and Routing Paths are verified using ping & traceroute.

8) When the main route fails, the backup route can be used for forwarding packets.

Diagram:



Thus, static routes were successfully configured and verified, including the use of a backup route when the main route becomes available.

11)b)

Date: 5.10.26

RIP - Dynamic Routing

AIM:

To simulate and configure RIP (Routing Information Protocol) using Cisco Packet Tracer.

Observation:

1) Three Routers are connected to form a network with multiple possible paths.

2) IP Addresses are assigned to the PC's, Fast Ethernet Interfaces and Serial Interfaces.

3) RIP is enabled using Vector rip command.

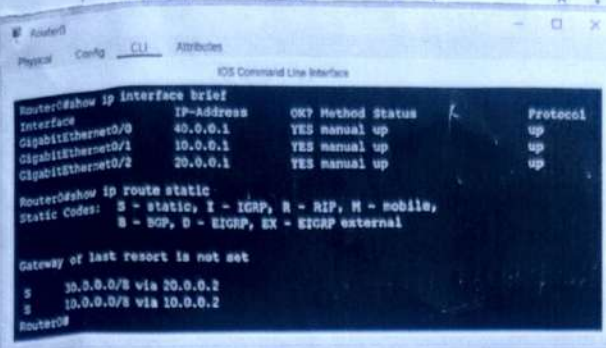
4) Directly connected networks are advertised using Network command.

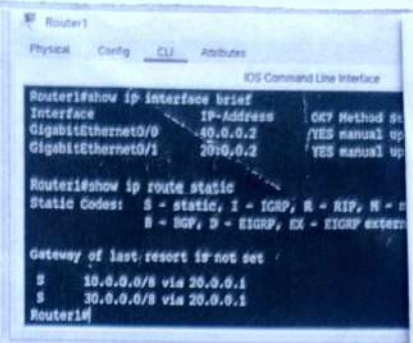
5) RIP automatically exchanges routing information between Routers.

6) RIP selects the Route based on the lowest hop count.

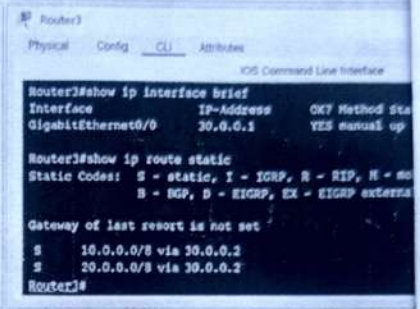
7) The ping command is used to test connectivity between PC0 and PC1.

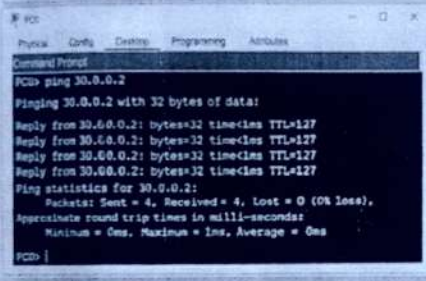
8) The transcript command is used to verify the

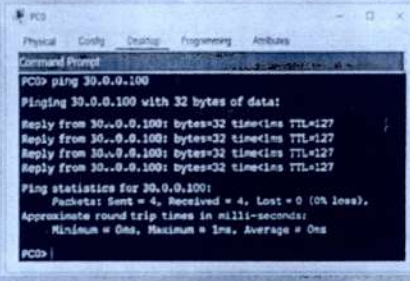


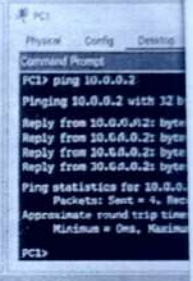


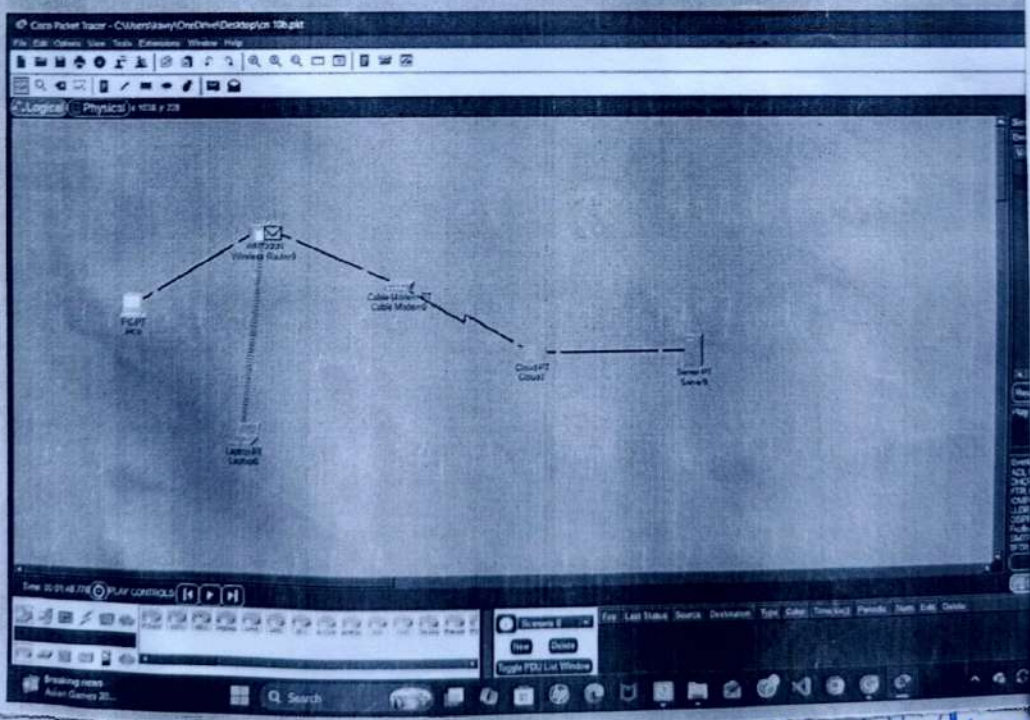












Ignored & dynamic

Routing ~~was~~ verified.

12)a)

Date:

End to End Communication at Transport Layer

a) Implement echo client server using TCP/UDP sockets

b) Implement a chat program using socket programming

TCP:

```
import socket
import threading

#--SERVER--
def start_server():
    server = socket.socket(socket.AF_INET,
                           socket.SOCK_STREAM)
    server.bind(('127.0.0.1',5000))
    server.listen(1)
    print("[SERVER] Listening on 127.0.0.1:5000")
    conn,addr = server.accept()
    print(f"[SERVER] Connected by {addr}")
while True:
    data = conn.recv(1024).decode()
    if not data or data.lower() == 'quit':
```

```
        print("[SERVER] Connection closed by client.")
        break
    Print(f"[SERVER] Connection closed by client.")
    break
    Conn.send(f"Server recieved: {data}".encode())
Conn.close()
server.close()

#-- CLIENT --
def start_client()
    client = socket.socket(socket.AF_INET, Socket.
                SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
    Print("[CLIENT] Connected to server")
while True:
    msg = input("Enter message (or 'quit' to exit:")
    client.send(msg.encode())
    if msg.lower() == 'quit':
        break
    response = client.recv(1024).decode()
```

```
        print(f"[SERVER RESPONSE] {response}")
    client.close()

# --- MAIN ---
if __name__ == "__main__":
    server_thread = threading.Thread(target=
                    start_server, daemon=True)
    server_thread.start()
    start_client()
```

Output:

Listening on 127.0.0.1:5000. [CLIENT]

Enter msg: hello

[SERVER] Recieved: hello

[SERVER RESPONSE] Server Recieved: hello

Enter message (or 'quit or Exit): quit

Connection closed by Client.

UDP

```
import socket
import threading

#--SERVER--
def udp_server():
    s=socket.socket(socket.AF_INET, Socket.
                    SOCK_DIAGRAM)
    s.bind(('127.0.0.1',12000))
    print("[SERVER] Running on 127.0.0.1:12000")
    while True:
        data,addr=s.recvfrom(1024)
        msg =data.decode()
        if msg.lower()='quit':
            print(f"[SERVER] Client {addr}
                  ended the Connection.")
            break
        print (f"[SERVER] Recieved from
               {addr}:{msg}")
        s.sendto(data,addr)# Echo back
    s.close()
    print("[SERVER] stopped.")
```

```
#-- CLIENT --
def udp-client():
    c = socket.socket(socket.AF_INET, socket.
        SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message (or 'quit' to
              stop): ")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[CLIENT] Closing
                  connection...") break
        data, _ = c.recvfrom(1024)
        print("Echo from server:", data.
              decode())
    c.close()
    print("[CLIENT] Stopped.")
#-- MAIN --
if __name__ == "__main__":
    threading.Thread(target=udp_server,
    daemon=True).start() udp_client()
```

OUTPUT:

Enter message (or 'quit' to stop): [SERVER]
Running on 127.0.0.1:12000] hi

[SERVER] Recieved from ('127.0.0.1', 49670): hi

Echo from server: hi

Enter Message (or 'quit' to stop): hello

[SERVER] Recieved from ['127.0.0.1', 49670]:
how are you

Echo from Server: howare you

Enter message (or 'quit' to stop): quit

[CLIENT] Stopped [SERVER] Stopped

Result:

Thus, the end to end communication at Transport Layer is executed the code.

Exp.no.13

Date:

Implement your own ping program

Code:

```
import os
import platform
import subprocess

def ping(host="google.com", count=4):
    param="-n" if platform.system().lower()
        ="windows" else "-c"
    command=["ping", param, str(count), host]

    try:
        output=subprocess.check_output
            (command, universal_newlines
                =True)
        print(output)

    except Exceptionase e:
        print(f"Ping failed:{e}")

If __name__=="__main__":
    ping("8.8.8.8")
```

Output:

Pinging 8.8.8.8 with 32 bytes of data:

Reply from 8.8.8.8 : bytes = 32 time = 6ms

TTL = 119

Reply from 8.8.8.8 : bytes = 32 time = 9ms

TTL = 119

Reply from 8.8.8.8 : bytes = 32 time = 5ms

TTL = 119

Reply from 8.8.8.8 : bytes = 32, time = 9ms

TTL = 119

Ping statistics for 8.8.8.8 :

Packets : Sent = 4, Recieved = 4, Lost = 0

(0 % loss)

Approximate round trip times in milli - seconds :

Minimum = 5ms, Maximum = 9ms, Average = 7ms

Output:

Thus, the ping program is implemented using python code.

S.no: 14

Date:

Code using RAW Sockets to implement packet sniffing

Aim:

TO write a code using RAW sockets to implement packet sniffing.

Code:

```
from scalpy.all import sniff
from scapy.layers.inet import TCP, IP,
UDP, ICMP

def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dest
    protocol_name = ""
    if protocol == 1:
        protocol_name = "ICMP"
    elif protocol == 6:
        protocol_name = "TCP"
```

```
elif protocol == 17:
    protocol_name = "UDP"
else
    protocol_name = "Unknown Protocol"

print(f"Protocol : {protocol_name}")
print(f"Source IP : {src_ip}")
print(f"Destination IP : {dst_ip}")
print("-"*50)

sniff(iface="Wi-Fi", prn=packet_callback,
      filter="ip", store=0)
```

Output :

Protocol : TCP

Source IP : 57.155.141.117

Destination IP : 192.168.1.6

Protocol : TCP

Source IP : 192.168.1.6

Destination IP : 57.155.141.117

INDEX

NAME: A.Avinash STD.: SEC.: ROLL NO.: 240701066 SUB.: CN observation

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1	23/7/26	Basic Networking Commands		
2)	27/7/26	Study of Different Types of Cables		
3)	27/7/26	CISCO PACKET TRACER		
4	03/08/26	SETTING UP A LOCAL NETWORK (LAN)		
5	10/8/26	EXPERIMENT ON PACKET CAPTURE TOOL WIRESHARK		
6	13/8/26	PYTHON Implementation of Hamming Code		
7	24/8/26	Sliding Window Protocol		
8	7/9/26	NMAP to Discover Live Host using Nmap Scans		
9]	17/9/26	Implementation of Subnetting in CISCO Packet Tracer		
10a)	24/9/26	Internetworking using Routers		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
10)b)	24.9.26	Wireless Router + DHCP + Internet		
11)a)	5.10.26	Static Routing		
11)b)	5.10.26	Dynamic Routing		

Completed